

# Java / Microservices / AWS - Lead Interview Bank

## Expanded Interviewer Edition

---

*Candidate profile: 18+ years | Theory-focused | Designed for a non-Java interviewer*

### How to use this edition

The original bank is intentionally concise. This edition keeps the same questions and reference answers, but adds interviewer-level explanation so you can understand why an answer is right, what trade-offs matter, and how to distinguish practical experience from memorized terminology.

For each question: ask the question, let the candidate answer without interruption, then use the reference answer only as a coverage guide. Follow with "Why?", "What trade-off did you accept?", and "Tell me about a production case." A lead does not need to match the wording below; a different answer can be strong if it is technically defensible and grounded in constraints.

Do not over-score jargon. Strong candidates explain mechanisms in plain language, quantify scale or impact, mention failure modes, and state when they would choose a simpler option.

## 1. Java and JVM

### Q1. Walk me through the main JVM memory areas. In production, which memory problems do you watch for, especially in containers?

#### Reference answer

The JVM divides memory into these areas: Heap — all objects. The garbage collector manages it. The heap has a young generation (Eden and two survivor spaces) and an old generation. Metaspace — class metadata. It is in native memory, not in the heap. Thread stacks — one stack for each thread. Local variables and call frames stay here. Code cache — machine code from the JIT compiler. Native / direct memory — buffers outside the heap. Most objects die young. The collector moves the survivors to the old generation. A full collection of the old generation is expensive. Common production problems: Heap exhaustion from a cache or a collection that never releases entries. Metaspace growth from dynamic class generation or from repeated redeployments. Native memory growth from direct byte buffers or from a native library.

#### Interviewer explanation - what this means

Think of JVM memory as several budgets, not one. The heap is only the object budget; metaspace, thread stacks, direct buffers and JVM/native structures also consume the process or container limit. This is why a pod can be OOMKilled even when heap usage looks healthy. A strong lead should reason from total resident memory and leave headroom rather than tuning only -Xmx.

#### How to judge the candidate

she separates heap from metaspace and from native memory. She knows that container memory limits count all three. A lead must say that you cannot set the container limit equal to the heap size.

### Useful follow-up

"Your pod restarts with OOMKilled, but the heap graph looks normal. What do you check?" (Expected: native memory, metaspace, thread count, direct buffers, -XX:MaxRAMPercentage .)

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

## Q2. How do you find the cause of a memory leak in a long-running service?

### Reference answer

A leak in Java is not a lost pointer. It is a reference that the application keeps but no longer needs. Method: 1. Confirm the leak. Watch the heap after each full collection. If the base level goes up over days, you have a leak. 2. Take a heap dump. Compare two dumps taken hours apart. 3. Find the dominator tree. It shows which object holds the most memory. 4. Find the reference path from the GC root to that object. Frequent causes: A static map that the code uses as a cache with no size limit. A ThreadLocal value in a pooled thread. The pool never removes the thread, so the value stays. Listeners or callbacks that nobody removes. Class loader leaks after a redeploy in an application server.

### Interviewer explanation - what this means

The important idea is retention, not allocation. High allocation can be healthy if GC reclaims it; a leak exists when live memory after GC keeps trending upward. The dominator tree tells you which retained object owns a large part of the graph, while the path to a GC root explains why it cannot be collected. Look for a methodical evidence-driven investigation rather than 'increase heap and restart'.

### How to judge the candidate

she uses "GC root" and "dominator". She knows tools such as heap dumps, Mission Control, or async-profiler. She checks a graph first. She does not restart and hope.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

## Q3. What are virtual threads? When do they help, and when do they not help?

### Reference answer

A platform thread maps to one operating-system thread. It is expensive. A server can hold some thousands of them. A virtual thread is a lightweight thread that the JVM schedules. Many virtual threads run on a few carrier threads. When a virtual thread blocks on I/O, the JVM removes it from the carrier thread. The carrier thread then runs other work. Virtual threads help when: The service is I/O bound. It waits for a database, a queue, or another service. You want simple blocking code but high concurrency. Virtual threads do not help when: The work is CPU bound. The number of cores still limits you. The bottleneck is downstream. More concurrency then makes the downstream failure worse. Warning: virtual threads remove the natural limit that a thread pool gave you. You must add your own limit, such as a semaphore, or you overload the database.

### Interviewer explanation - what this means

Virtual threads improve concurrency for blocking I/O by making waiting threads cheap; they do not make CPU work faster. The architectural risk is that cheap concurrency can send far more simultaneous calls to a database or downstream service. The candidate should therefore pair virtual threads with explicit concurrency limits, timeouts and downstream capacity thinking.

### How to judge the candidate

she says that virtual threads are not a performance trick. They replace complex reactive code with simple code. A strong lead mentions the loss of built-in backpressure.

### Useful follow-up

"Do virtual threads make reactive frameworks unnecessary?" (Good answer: for most request/response services, yes. For streaming and for operators such as merge or throttle, no.)

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

## Q4. Why must equals and hashCode agree? What breaks if they do not?

### Reference answer

hashCode gives an integer for an object. Hash-based collections use it to select a bucket. equals compares two objects for logical equality. The contract: If two objects are equal, they must give the same hash code. If two objects give the same hash code, they are not always equal. If you break the contract, a hash map loses entries. You put an object in the map. You then look for an equal object. The map calculates a different bucket and finds nothing. A second failure happens if you change a field after you put the object in a set. The hash code changes. The object stays in the old bucket. The set now holds an item that it cannot find.

### Interviewer explanation - what this means

Hash collections first use hashCode to narrow the search to a bucket and then equals to establish logical equality. Equal objects with different hashes can therefore become effectively invisible to lookups. Mutable keys are especially dangerous because changing a field used by equals/hashCode after insertion changes where the object logically belongs without moving it in the map.

### How to judge the candidate

she mentions mutable keys. That is the failure that hurts in production. Records and immutable value objects prevent it.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

## Q5. Immutability — why does a lead push for it?

### Reference answer

An immutable object does not change after construction. Benefits: It is thread safe with no lock. You can share it and cache it safely. It is a safe map key. Debugging is simpler, because the state at creation is the state at failure. Cost: Each change creates a new object. This adds allocation. Modern collectors handle short-lived objects well, so the cost is usually small. Large object graphs need builder patterns or copy methods. Java gives records for simple data carriers. Records give the constructor, accessors, equals, hashCode, and toString.

### Interviewer explanation - what this means

Immutability removes an entire class of shared-state bugs because readers never observe an object halfway through mutation. It also makes value semantics, caching and reasoning easier. Do not reward absolutism: large mutable working structures can be appropriate inside a confined scope, and a Java record is only shallowly immutable if it contains mutable collections.

### How to judge the candidate

she treats immutability as a default and not as a religion. She knows that a record with a mutable list inside is not really immutable.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

### Q6. Which Java version would you target for a new service today, and how would you approach upgrading a large Java 8/11 estate?

#### Reference answer

Java 17 and Java 21 are long-term-support versions. Java 21 gives virtual threads, pattern matching for switch, sealed classes, and records. A lead answer includes the migration reason: Security patches for old versions stop. New collectors give shorter pauses. Library versions and Spring Boot 3 need Java 17 as a minimum. Newer versions give better container support and lower memory use.

#### Interviewer explanation - what this means

The version number matters less than the upgrade discipline. A lead should discuss LTS support, framework compatibility, security, performance and a staged migration with testing and rollback. For an old estate, ask about dependency upgrades, javax-to-jakarta migration, illegal reflection, removed APIs and production rollout rather than accepting 'we use Java 21'.

### How to judge the candidate

she gives an upgrade plan and not only a version number. Ask how she moved a large estate from Java 8 or 11. A real answer includes removed APIs, javax to jakarta package changes, and reflection restrictions. 2. Concurrency

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

## 2. Concurrency

### Q7. What does volatile guarantee, and what does it not guarantee?

#### Reference answer

volatile gives visibility and ordering. A write to a volatile field becomes visible to all other threads that read the field after it. The compiler and the processor cannot move other operations across it in a way that breaks this rule. volatile does not give atomicity for compound operations. `count++` is a read, an add, and a write. Two threads can lose an update, even with volatile. For a counter, use an atomic class or a lock. For a flag that one thread sets and other threads read, volatile is correct and cheap.

#### Interviewer explanation - what this means

volatile establishes visibility and a happens-before relationship for reads/writes of that variable, but it does not turn multi-step logic into one indivisible operation. `count++` can still race because two threads may read the same old value. A flag is a classic good use; counters or check-then-act logic usually need atomics, locks or a different design.

#### How to judge the candidate

she says "visibility, not atomicity" quickly. Then she gives the counter example.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

### Q8. How would you size and configure a thread pool for a production service?

#### Reference answer

The correct size depends on the type of work. CPU-bound work: the pool size is near the number of available cores. More threads only add context switches. I/O-bound work: the pool can be larger. A common start point is cores multiplied by  $(1 + \text{wait time} / \text{service time})$ . More important than the formula: Use a bounded queue. An unbounded queue hides overload until the service runs out of memory. Define the rejection policy. The caller must get a fast failure, not an infinite wait. Use separate pools for different downstream systems. This is the bulkhead pattern. A slow payment provider must not consume all threads and stop the login path. In a container, check that the JVM sees the correct core count.

#### Interviewer explanation - what this means

Pool sizing is capacity control, not a magic formula. CPU work is bounded by cores; I/O work can tolerate more concurrency because threads spend time waiting. The more senior answer covers queue bounds, rejection, isolation between dependencies, measurement under load and the total concurrency imposed on downstream systems.

#### How to judge the candidate

she talks about the queue and the rejection policy, not only the number. That difference separates a lead from a senior developer.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

## Q9. Explain deadlock. How do you find one in production?

### Reference answer

A deadlock happens when two threads each hold a lock that the other thread needs. Neither thread can continue. Four conditions must be true together: mutual exclusion, hold and wait, no preemption, and circular wait. If you remove one condition, you remove the deadlock. Prevention: Always take locks in the same global order. Use a lock timeout, and fail instead of waiting. Reduce the size of the critical section. Prefer immutable data and message passing over shared state. Detection: take a thread dump. The JVM reports "Found one Java-level deadlock" and names the threads and the locks.

### Interviewer explanation - what this means

A deadlock is permanent circular waiting, not merely a slow lock. In Java, a thread dump can show the cycle directly; in databases, deadlocks often arise when transactions lock rows in inconsistent order. Strong answers move from detection to prevention: consistent lock ordering, shorter transactions, timeouts and less shared mutable state.

### How to judge the candidate

she also mentions distributed deadlock, such as two database transactions that lock rows in a different order. That is a more frequent production problem than a Java-level deadlock.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

## Q10. What is backpressure, and where do you apply it?

### Reference answer

Backpressure is the signal from a slow consumer to a fast producer. It tells the producer to slow down. Without backpressure, work collects in a queue. Latency grows. Memory grows. The system fails completely instead of degrading. Places to apply it: Bounded queues in thread pools. Connection pool limits and acquisition timeouts. Rate limits at the API gateway. Consumer prefetch limits in a message broker. Load shedding: reject low-priority requests early when the system is near its limit.

### Interviewer explanation - what this means

Backpressure keeps demand from growing without bound when a consumer is slower than a producer. Without it, queues hide overload until latency and memory explode. Look for controls at multiple layers - bounded queues, pool limits, rate limits, broker settings and load shedding - plus awareness that aggressive retries can defeat those controls.

### How to judge the candidate

she says that a timeout is a form of backpressure, and that a retry without a limit removes it. Ask her how a retry storm starts. 3. Spring Boot

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

### 3. Spring Boot

#### Q11. How does Spring auto-configuration work?

##### Reference answer

A starter is a dependency that brings other dependencies and configuration classes. At startup, Spring Boot reads a list of auto-configuration classes from the file `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`. Older versions used `spring.factories`. Each configuration class carries conditions. Examples: apply this class only if a class is on the classpath, only if the user did not define this bean, or only if a property has a value. Spring evaluates the conditions and creates only the beans that pass. Your own beans win. Auto-configuration runs after user configuration and steps back if a bean already exists.

##### Interviewer explanation - what this means

Auto-configuration is conditional configuration, not magic. Spring Boot checks what is on the classpath, which properties are set and which beans the application already provides, then activates matching configuration. A lead should know how to inspect the condition evaluation report and how this mechanism enables internal starters without preventing application-level overrides.

##### How to judge the candidate

she knows the conditional mechanism and the report you get with `-- debug`. Ask her to describe how she built an internal starter for shared company concerns, such as logging or security. That is a lead-level activity.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

#### Q12. Why can a method annotated with `@Transactional` execute without a transaction? Give the common Spring proxy cases.

##### Reference answer

Spring applies transactions with a proxy. The caller talks to the proxy. The proxy opens the transaction and then calls your object. The annotation has no effect when: One method of the same class calls another method directly. The call does not pass through the proxy. This is self-invocation. The method is private or final, so the proxy cannot override it. The call comes from a different thread. The class is not a Spring bean. Also, the default rollback rule applies only to unchecked exceptions. If you catch the exception, or if you throw a checked exception without `rollbackFor`, the transaction commits.

##### Interviewer explanation - what this means

The key concept is interception. `@Transactional` works when a call crosses the Spring proxy that surrounds the bean; an internal `this.method()` call bypasses that proxy, so no transaction advice runs. Also judge whether the candidate understands rollback rules and transaction boundaries, not just the self-invocation trivia.

##### How to judge the candidate

self-invocation. Every experienced Spring developer has lost a day to it. If she cannot explain the proxy, she has used Spring but has not debugged it.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

### Q13. Explain transaction propagation. Give a case for REQUIRES\_NEW.

#### Reference answer

Propagation defines what happens when a transactional method calls another transactional method. REQUIRED (default): join the current transaction, or start one if none exists. REQUIRES\_NEW : suspend the current transaction and start a separate one. It commits or rolls back on its own. SUPPORTS : join if a transaction exists, else run without one. MANDATORY : fail if no transaction exists. NESTED : use a savepoint inside the current transaction. Use REQUIRES\_NEW when a step must survive the failure of the caller. Example: write an audit record or a failure record. The main transaction rolls back, but the audit row stays. Risk: REQUIRES\_NEW takes a second connection from the pool while the first one stays open. With a small pool and high load, this causes a deadlock in the pool.

#### Interviewer explanation - what this means

Propagation controls how nested service calls share or create transaction contexts. REQUIRES\_NEW is independent, which can be useful for audit/failure records, but it suspends the outer transaction and usually needs another database connection. Under load that can exhaust the pool, so a lead should use it deliberately rather than as a generic fix.

#### How to judge the candidate

the connection pool risk. Few people mention it. It is a real production failure.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

### Q14. Spring MVC or WebFlux — how do you choose?

#### Reference answer

Spring MVC uses one thread for each request. The thread waits during I/O. It is simple to write and simple to debug. WebFlux is non-blocking. A small number of threads handle many connections. It suits high connection counts, streaming, and services that mostly wait on other services. Costs of WebFlux: The programming model is harder. Stack traces are hard to read. One blocking call in the chain destroys the benefit. The team must learn it. Every library in the path must be non-blocking. Since Java 21, virtual threads give much of the throughput benefit with the simple blocking model. For most request/response services, MVC with virtual threads is now the better choice. WebFlux still wins for streaming and for complex flow control.

#### Interviewer explanation - what this means

The decision is about workload and operational complexity. MVC is straightforward and works well for ordinary request/response services; WebFlux is valuable when the entire path is non-blocking and the workload needs streaming or very high concurrent connections. With virtual threads, blocking MVC can scale much further while preserving simpler code, so the candidate should justify reactive complexity rather than treating it as modern by default.

#### How to judge the candidate

she does not choose reactive by default. She names the team cost. A lead thinks about who maintains the code at 02:00.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

## Q15. What would you put in place before calling a Spring Boot service production-ready?

### Reference answer

Health: liveness and readiness endpoints. Readiness must fail when a critical dependency is down, so the load balancer stops sending traffic. Metrics: Micrometer with request rate, error rate, and latency percentiles. Watch p99, not the average. Tracing: one trace identifier through all services and all logs. Logging: structured JSON. No personal data. Correct levels. Configuration: outside the artifact. Secrets in a secret store, not in properties files. Graceful shutdown: stop new requests, finish current work, then exit. Timeouts: on every outbound client. A default of "no timeout" is a fault. Resource limits: connection pool size, memory percentage for the container.

### Interviewer explanation - what this means

Production readiness means the service can be observed, controlled and safely operated during failures and deployments. Health checks, metrics, traces, structured logs, external configuration, secret management, timeouts, graceful shutdown and resource limits form one operational system. A strong candidate distinguishes liveness from readiness and discusses SLO-relevant latency percentiles rather than averages alone.

### How to judge the candidate

the difference between liveness and readiness. Many people configure the same check for both. That causes restart loops during a downstream outage. 4. Microservices and distributed systems

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

## 4. Microservices and distributed systems

### Q16. How do you decide the boundary of a service?

#### Reference answer

Split by business capability, not by technical layer or by database table. Signals of a good boundary: The service owns its data. No other service reads its tables. One team can change it and release it alone. Most changes touch one service only. The language of the domain is consistent inside the boundary. Signals of a bad boundary: Every feature needs a change in three services. Services share a database schema. A "customer service" exists that only performs create, read, update, and delete on one table. It has no behaviour. Chatty communication. One request causes twenty internal calls.

#### Interviewer explanation - what this means

A service boundary should align with a cohesive business capability and ownership boundary. If every change requires coordinated releases or services share tables, the boundary is probably wrong. Look for bounded-context thinking, independent data ownership and willingness to merge services when decomposition created excessive coupling.

#### How to judge the candidate

"bounded context", "team ownership", and "we merged two services back together". The last one shows real experience. Splitting is easy. Correcting a split is hard.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

### Q17. How do you keep data consistent across services without a distributed transaction?

#### Reference answer

Two-phase commit does not scale and couples availability. Use a saga. A saga is a sequence of local transactions. Each step publishes an event. If a step fails, the saga runs compensating actions to undo the previous steps. Two styles: Choreography: each service listens to events and reacts. It is simple for a few steps. It becomes hard to follow when the flow grows, because no place shows the whole process. Orchestration: one coordinator tells each service what to do. The flow is visible and testable. The coordinator is one more component to run. Important: a compensation is not a rollback. You cannot un-send an email. You send a correction. The business must define what the correction is.

#### Interviewer explanation - what this means

Cross-service consistency is usually a business workflow, not one ACID transaction. A saga commits local steps and uses compensating actions when later steps fail; compensation may be a new business action rather than a literal rollback. Strong candidates discuss idempotency, observability and whether choreography or orchestration keeps the workflow understandable.

#### How to judge the candidate

she says that the business must define compensation. That is the true lead answer.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

## Q18. You must update the database and publish an event. How do you avoid losing one of them?

### Reference answer

This is the dual-write problem. A commit to the database and a publish to a broker are two separate operations. A failure between them leaves the system inconsistent. Solution: the transactional outbox. 1. In the same database transaction, write the business row and an outbox row. 2. A separate process reads the outbox and publishes the message. 3. The process marks the row as sent, or deletes it. The reader can poll the table, or it can read the database log with change data capture. This gives at-least-once delivery. The consumer can receive the same message twice. The consumer must therefore be idempotent, usually with a message identifier and a table of processed identifiers.

### Interviewer explanation - what this means

The problem is atomicity across two systems: the database can commit while publishing fails, or vice versa. The outbox makes the business change and the intent-to-publish atomic in one database transaction, then publishes asynchronously. Because publication may repeat, the consumer must tolerate duplicates; that is why outbox and idempotency belong together.

### How to judge the candidate

"outbox" and "idempotent consumer" together. If she claims exactly- once delivery across systems, ask how. Exactly-once processing exists only as at- least-once delivery plus deduplication.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

## Q19. How would you design a resilient call from one service to an unreliable downstream dependency?

### Reference answer

Layers, in order: 1. Timeout — always. The timeout must be shorter than the caller's own budget. 2. Retry — only for safe, idempotent operations. Use a limited count. Use exponential delay with random jitter. Jitter stops all clients from retrying at the same moment. 3. Circuit breaker — after a number of failures, stop calling. Fail fast. Test with a small number of trial calls before you close the circuit again. 4. Bulkhead — a separate connection pool or thread pool for each dependency. 5. Fallback — cached data, a default value, or a clear degraded response. Danger: retries multiply load. If three services each retry three times, the last service receives 27 times the traffic. This turns a small problem into an outage. Retry at one level only, and give each request a deadline that all levels respect.

### Interviewer explanation - what this means

Resilience should be designed around an end-to-end latency/deadline budget. Timeouts stop indefinite waiting; retries are selective and bounded; circuit breakers stop repeated calls during failure; bulkheads contain blast radius; fallbacks define degraded behavior. The lead-level insight is that retries multiply load and can convert a partial outage into a full one.

### How to judge the candidate

the retry amplification effect and the deadline idea. This is the difference between a person who configured Resilience4j and a person who has handled an outage.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

## Q20. How would you evolve/version an API used by many independent teams without breaking consumers?

### Reference answer

Rule: additive changes only. Add optional fields. Never remove or rename a field. Never change the meaning of a field. For breaking changes: Run both versions at the same time. Publish a deprecation date and measure which client still calls the old version. Move consumers one by one. Remove the old version only when the metrics show zero traffic. For events, the same rule applies. Use a schema registry with a compatibility check in the build pipeline. The build must fail on an incompatible schema. Consumer-driven contract tests stop the accidental break. Each consumer publishes its expectations. The producer's pipeline runs those expectations.

### Interviewer explanation - what this means

Compatibility is an operational lifecycle, not just a URL version. Additive changes preserve old consumers; breaking changes require coexistence, migration evidence and an explicit retirement plan. Strong candidates measure old-client traffic and apply the same compatibility discipline to event schemas and consumer contracts.

### How to judge the candidate

she measures old-version traffic before removal. Anybody can add "/v2". Retiring "/v1" is the real work.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

## Q21. A user request crosses eight services and is failing or slow. How would you diagnose it in production?

### Reference answer

Correlation: one trace identifier created at the edge and passed to every call and every log line. Distributed tracing: spans show where the time goes. Sampling keeps the cost low, but always keep the traces of errors and slow requests. Metrics first, logs second: metrics tell you which service is unhealthy. Logs tell you why. Percentiles: watch p95 and p99. An average hides the problem. Service dependency map: you must know who calls whom before the incident, not during it. Common trap: high cardinality labels on metrics, such as user identifier. This makes the metrics system fail.

### Interviewer explanation - what this means

For a multi-service incident, observability must let you narrow the fault quickly: metrics identify where, traces show the path and timing, and logs explain details. Correlation IDs and trace context must propagate across boundaries. A lead should discuss cardinality, sampling, dependency maps and concrete detection/diagnosis times from incidents.

### How to judge the candidate

a real incident story with a timeline. Ask: "What was the last incident you led? How long did detection take?" Detection time shows the quality of the observability.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

## Q22. When should a team NOT use microservices?

### Reference answer

Do not use them when: The team is small. One team with five people does not need eight deployment units. The domain is not yet clear. A wrong boundary in a distributed system is very expensive to correct. There is no automated pipeline, no monitoring, and no on-call process. Microservices multiply operational work. The load is low. The complexity gives no benefit. A modular monolith is often the correct first step. It gives clear internal boundaries with one deployment. You can extract a module later if a real need appears, such as an independent scaling requirement or a separate team.

### Interviewer explanation - what this means

Microservices buy independent deployment, scaling and ownership at the cost of network failure modes, operational tooling and coordination. If the team/domain is small or immature, a modular monolith often gives most structural benefits with far less operational cost. Reward candidates who choose architecture from constraints instead of ideology.

### How to judge the candidate

she is willing to say "monolith". A lead who cannot argue against microservices will build a distributed monolith.  
5. Data

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

## 5. Data

### Q23. Optimistic or pessimistic locking — which one and when?

#### Reference answer

Pessimistic locking takes a database lock when it reads. Other transactions wait. It suits short transactions with high contention on the same row, such as a stock count in a flash sale. Optimistic locking uses a version column. The update includes the expected version. If the version changed, the update affects zero rows and the application retries. It suits low contention and long user interactions, because it holds no lock while a person thinks. Pessimistic risk: lock waits, and deadlocks if transactions take rows in a different order. Optimistic risk: a high retry rate when contention is high. The retries waste work.

#### Interviewer explanation - what this means

Pessimistic locking prevents conflicting work by blocking early; optimistic locking allows concurrency and detects a conflict at write time. The choice depends mainly on contention, transaction duration and the cost of retry. A lead should recognize that optimistic locking under heavy contention can become a retry storm, while pessimistic locking can create waits and deadlocks.

#### How to judge the candidate

she connects the choice to contention level and transaction length, not to personal preference.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

### Q24. What is the N+1 query problem, and how do you detect it?

#### Reference answer

The application runs one query for a list of parents. Then, for each parent, it runs one more query for the children. One hundred parents cause 101 queries. Cause: lazy loading in an object-relational mapper. The code looks correct. Each access to a collection triggers a hidden query. Fixes: a join fetch, an entity graph, a batch size setting, or a separate query for all children at one time. Detection: count the queries in a test, log the statements in a development environment, or use a tracing tool that shows database spans. A lead adds an automatic check that fails the build when a request runs too many queries.

#### Interviewer explanation - what this means

N+1 is dangerous because ORM code can hide network/database round trips behind innocent-looking property access. The performance problem grows with result size and may appear only in production data volumes. Strong candidates can fix it, but leads also put query-count tests, tracing or review rules in place so it does not recur.

#### How to judge the candidate

the automatic check. Everybody can fix one N+1 case. A lead stops the next one.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

## Q25. How would you size a database connection pool across multiple application instances?

### Reference answer

The pool must be small. A common result is around ten connections for each instance. Reason: the database performs work with its cores and its disks. More connections than the database can serve only add queue time and context switches. Throughput does not go up. Latency goes up. Rules: Multiply the pool size by the number of instances. The total must stay under the database limit. Set an acquisition timeout. A request must fail fast when no connection is free. Keep transactions short. Never call another service inside an open transaction. Watch the wait time for a connection. That metric shows the problem before an outage.

### Interviewer explanation - what this means

A database connection is not free parallelism; too many concurrent queries can make the database slower. Pool sizing must consider database capacity, number of application instances, transaction duration and acquisition wait time. The particularly important smell is holding a DB connection/transaction while waiting on a remote service.

### How to judge the candidate

"never call a remote service inside a transaction". This is a frequent cause of pool exhaustion.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

## Q26. Explain a cache strategy and its failure modes.

### Reference answer

Cache-aside is the usual pattern. The application reads the cache. On a miss, it reads the database and then writes the cache. Failure modes: Stampede: a popular key expires. Many requests miss at the same time and hit the database. Fix: a lock for the refresh, or an early refresh of hot keys. Synchronised expiry: many keys expire together. Fix: add random jitter to the time-to-live value. Stale data: the database changes but the cache does not. Fix: invalidate on write, or accept a short, defined staleness. Cache as a database: the system cannot work when the cache is empty. A restart then causes an outage. The service must survive a cold cache.

### Interviewer explanation - what this means

A cache trades freshness and simplicity for lower latency/load. Cache-aside is common, but expiry and failure behavior must be designed: stampedes, synchronized expiry, stale values and cold starts can all overload the source database. A lead should state what correctness level the cache is allowed to violate and what happens when the cache is unavailable or empty.

### How to judge the candidate

the cold-start question. Ask her: "Your cache cluster restarts empty at peak time. What happens?" Her answer shows whether the cache is an optimisation or a hidden dependency. 6. AWS

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

## 6. AWS

### Q27. For a Java workload on AWS, how would you choose between Lambda, ECS/Fargate, EKS and EC2?

#### Reference answer

Lambda: event work, spiky load, short tasks. You pay for each call. There is no server to patch. Limits: execution time, cold start, and no long-lived connections. ECS on Fargate: containers with no node management. Good for standard services. Less control and higher cost for each unit than EC2. EKS: Kubernetes. Choose it when you need the ecosystem, complex workloads, or portability. It has a real operational cost. It needs a platform team. EC2: full control, special hardware, or licences that need a fixed host. Decision factors: traffic shape, cold-start tolerance, team skill, and the total cost including people. For a Java service, cold start matters. A JVM on Lambda starts slowly. Options: SnapStart, provisioned concurrency, or a native image. A lead often answers that a steady-load Java service belongs in a container, and that Lambda suits the event edges.

#### Interviewer explanation - what this means

These compute choices trade operational control against abstraction. Lambda is strong for event-driven/spiky short work; Fargate for managed containers; EKS when Kubernetes capabilities justify a platform burden; EC2 when host-level control is required. A lead should include traffic shape, startup behavior, networking, team skills and total cost of ownership, not just service features.

#### How to judge the candidate

she includes the cost of operating people, not only the price of the service.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

### Q28. How would you choose among SQS, SNS, EventBridge, Kinesis and MSK/Kafka for a messaging requirement?

#### Reference answer

SQS: a queue between two components. One consumer group takes each message. It gives retry, a visibility timeout, and a dead-letter queue. FIFO queues give order inside a message group, with lower throughput. SNS: publish and subscribe for fan-out to several endpoints. No storage or replay. EventBridge: an event bus with content-based rules, schema registry, and many software-as-a-service integrations. It suits domain events and routing. Kinesis: an ordered stream with retention and replay. Consumers keep their own position. It suits high-rate telemetry and analytics. MSK (Kafka): like Kinesis but Kafka, with a large ecosystem and long retention. It suits event sourcing and stream processing. The operational cost is higher. Key questions to decide: do you need order, do you need replay, do you need fan-out, and what is the message rate?

#### Interviewer explanation - what this means

Choose messaging based on semantics: queue vs fan-out vs event routing vs replayable stream, plus ordering and throughput. SQS distributes work, SNS fans out, EventBridge routes events, Kinesis provides managed ordered streams, and MSK provides Kafka semantics/ecosystem. The candidate should reason about retries, DLQs, replay, consumer position and poison messages rather than naming products from memory.

### How to judge the candidate

she chooses on order and replay, not on popularity. Ask about the SQS visibility timeout and the poison-message loop. A wrong visibility timeout causes duplicate processing.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

## Q29. When do you choose DynamoDB, and what is the main design risk?

### Reference answer

Choose DynamoDB when the access patterns are known, stable, and simple, and when you need predictable latency at a large scale. You design the table from the queries, not from the entities. You may keep several entity types in one table. Main risk: the partition key. All items with the same key live in one partition. A bad key creates a hot partition. That partition throttles while the rest of the table is idle. Example: a date as the key sends all of today's traffic to one partition. Fix: add a suffix or another attribute to spread the load. Second risk: a new query that the key design does not support. You then add a global secondary index, which has its own capacity and eventual consistency, or you migrate the data. Use a relational database when the queries are not predictable, when you need joins and reports, or when the team must explore the data.

### Interviewer explanation - what this means

DynamoDB is designed from access patterns backward. A poor partition key can concentrate traffic into a hot partition even when overall capacity is large, and new query patterns may require indexes or redesign. Strong candidates ask what queries must be fast, expected key distribution and consistency needs before discussing table shape.

### How to judge the candidate

hot partitions and "access patterns first". If she designs DynamoDB tables like relational tables, she has only used it lightly.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

## Q30. How would you design an AWS service to survive an Availability Zone failure? When would you also design for Region failure?

### Reference answer

Start with the business requirement. Ask for the recovery time objective and the recovery point objective. These two numbers decide the cost. Multi-availability-zone: standard. Instances in at least three zones. A database with a synchronous standby in a second zone. Recovery is automatic and cheap. Multi-region, warm standby: data replicates to a second region. A small copy of the stack runs there. Failover takes minutes and needs a decision and a traffic change. Multi-region, active-active: both regions serve traffic. It gives the best recovery, and it is the most expensive and the most complex. Write conflicts become a real problem. You must define the conflict rules in the domain. Also required: a tested runbook. An untested failover plan is not a plan. Ask when she last ran a real failover test.

### Interviewer explanation - what this means

Disaster-recovery design starts with RTO (how long can we be down?) and RPO (how much data can we lose?). Multi-AZ handles infrastructure failure within a region; multi-region strategies address regional failure but add

replication, failover and conflict complexity. The strongest signal is evidence of tested failover, not a diagram that has never been exercised.

#### **How to judge the candidate**

she asks for the objectives before she designs. That is the correct first move.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

### **Q31. How would you control and continuously reduce AWS cost across a microservice estate?**

#### **Reference answer**

Tag everything, then show the cost for each team and each service. Visibility changes behaviour. Right-size. Most services use far less CPU than they request. Use Graviton processors for standard workloads. The price for each unit of performance is lower. Use savings plans for the steady base load. Use on-demand or spot for the variable part. Watch data transfer and NAT gateway charges. Cross-zone traffic and NAT are frequent hidden costs. Set log retention. Log storage grows without limit if nobody sets a rule. Delete unused storage volumes, snapshots, old images, and idle load balancers.

#### **Interviewer explanation - what this means**

Cloud cost is an architecture metric. Tagging/chargeback gives ownership; rightsizing and commitment discounts reduce compute cost; network/NAT/log/storage costs often hide outside CPU graphs. A lead should be able to name a concrete optimization, its measured saving and any reliability/performance trade-off.

#### **How to judge the candidate**

a specific number. "We reduced the monthly bill by 30 percent by doing X." A lead has done this work, because finance asked.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

### **Q32. How would you manage application secrets and AWS access in production?**

#### **Reference answer**

No long-lived keys in code, in images, or in environment files in the repository. Give each workload a role. On EKS, give the pod a role through service accounts. On ECS, use a task role. Store secrets in Secrets Manager or in Parameter Store. Rotate them automatically. Give least privilege. Start with nothing and add what fails. Separate accounts for each environment. A development account must not reach production data. Log all access and alert on unusual actions.

#### **Interviewer explanation - what this means**

The default should be short-lived identity through workload roles, not distributed access keys. Secrets still need storage, controlled retrieval, least privilege, rotation and audit. Strong candidates separate environments/accounts and think about blast radius, emergency access and detection of anomalous use.

#### **How to judge the candidate**

she says "role", not "access key". She also mentions rotation. Many teams store the secret correctly and then never change it. 7. Kiro and AI-assisted delivery She lists Kiro. Test her judgement, not her tool knowledge.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

## 7. Kiro and AI-assisted delivery

### Q33. Explain spec-driven development in Kiro. How does it differ from prompt-based coding?

#### Reference answer

Kiro is an agentic development environment from AWS, built on the Code OSS base. It replaces Amazon Q Developer as the AWS developer tool. The workflow has three phases: 1. Requirements — you describe a feature. The tool produces user stories and acceptance criteria, in a structured notation, plus edge cases. 2. Design — the tool produces a technical design from the requirements. 3. Tasks — the tool produces a sequence of implementation tasks, and then the code. Two more parts: Steering files hold team standards, so the agent follows them in every task. Agent hooks run on events, such as a file save. They can update tests, refresh documents, or run a security scan. The difference from prompt-based coding: the specification is the artifact of record. The code becomes an output of the specification. You review the specification in version control and you can trace the code back to a requirement.

#### Interviewer explanation - what this means

Spec-driven development makes requirements/design/tasks durable artifacts that guide generation, rather than relying on a sequence of ephemeral prompts. The leadership question is whether the specification is reviewed, versioned and traceable to implementation and tests. Judge governance and repeatability more than familiarity with Kiro UI commands.

#### How to judge the candidate

she knows that specifications go into source control and into review. If she only says "it writes code faster", she has used it as a toy.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

### Q34. What are the risks of agent-generated code in a large team, and how do you control them?

#### Reference answer

Risks: False confidence. The code compiles and looks correct, but the behaviour at the edges is wrong. Review collapse. Output volume grows. Review capacity does not. Quality then falls. Silent duplication. The agent writes a new helper instead of using the existing one. Duplication grows. Security and licence problems. Copied patterns can carry known weaknesses. Unclear ownership. "The agent wrote it" is not an owner. Controls: The same review standard for all code. The author owns the output, always. Automatic gates: tests, static analysis, dependency scanning, and coverage in the pipeline. Shared steering rules, so all agents follow the same standards. Small changes. A large generated change cannot get a real review. Test the acceptance criteria, not the implementation.

#### Interviewer explanation - what this means

AI can increase output faster than a team can review it, so the bottleneck may shift from coding to validation. Risks include plausible-but-wrong behavior, duplicated patterns, security/licensing issues and weakened ownership. Strong controls combine human accountability, small reviewable changes, automated quality gates, shared standards and tests tied to acceptance criteria.

#### How to judge the candidate

"the human author owns it". A lead states this rule clearly.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

### Q35. How would you prove whether an AI coding tool is actually improving team delivery?

#### Reference answer

Do not measure lines of code or the number of accepted suggestions. Those numbers go up even when quality goes down. Better measures: Cycle time from start to production. Change failure rate and time to restore service. Defects that reach production. Review time and review queue length. Developer report of friction, collected regularly. Compare a period before and after, and keep the same team. Take care with the claim of causality, because many things change at the same time.

#### Interviewer explanation - what this means

Productivity must balance speed and quality. Cycle time alone can improve while production defects or review queues worsen, so pair delivery metrics with change failure rate, defects and recovery time. A mature answer also acknowledges confounding factors and uses controlled or at least comparable before/after measurement.

#### How to judge the candidate

she names a quality measure next to a speed measure. Speed alone is not a benefit. This links directly to your dual-estimation work, so her answer here tells you if she can support that programme.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

### Q36. When do you tell a developer NOT to use the agent?

#### Reference answer

Exploratory work, or a research spike. The structure adds friction with no gain. A small, well-understood change. Writing the specification takes longer than the change. Deep debugging of a rare production fault. This needs system knowledge, not generation. Sensitive areas, such as cryptography, payment logic, or a security boundary. Here you want a person who can defend every line. Junior developers who must learn the base concept. The tool removes the learning.

#### Interviewer explanation - what this means

The best use policy is risk- and context-based, not universal. AI may add little value to tiny changes or deep incident debugging, and high-risk domains require stronger human understanding and review. A lead should define where the tool accelerates work and where it creates unacceptable verification cost or learning loss.

#### How to judge the candidate

she gives a limit. A lead who says "use it for everything" has not led the review load. 8. Data structures and algorithms — verbal No code. Ask her to explain and to choose. Judge the reasoning.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

## 8. Data structures and algorithms - verbal

### Q37. HashMap or TreeMap — how do you choose, and when does a HashMap become slow?

#### Reference answer

A hash map gives constant average time for get and put. It has no order. A tree map gives logarithmic time and keeps the keys sorted, so it supports range queries and "the next key after X". A hash map becomes slow when: Many keys land in the same bucket, because the hash function is poor. Modern implementations change a long bucket chain into a tree, so the worst case becomes logarithmic instead of linear. The map grows and resizes. The resize copies all entries. If you know the size, set the capacity at construction. Someone uses a mutable key.

#### Interviewer explanation - what this means

HashMap optimizes average lookup; TreeMap trades speed for sorted order and range operations. Collision behavior, resizing and key immutability matter for HashMap performance/correctness. A lead should also know that concurrent access needs an appropriate concurrent structure rather than a plain HashMap.

#### How to judge the candidate

she mentions collision behaviour and initial capacity. Then ask about concurrent maps: a plain hash map used by several threads can corrupt itself.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

### Q38. Explain how an LRU cache works internally.

#### Reference answer

An LRU cache removes the entry that nobody used for the longest time. The structure combines two things: A hash map from key to node. It gives constant-time lookup. A doubly linked list in use order. The most recent entry is at the head. The oldest entry is at the tail. On access, you move the node to the head. On insert, when the cache is full, you remove the tail node and its map entry. Both operations are constant time, because the map gives direct access to the node.

#### Interviewer explanation - what this means

LRU needs  $O(1)$  lookup and  $O(1)$  recency updates, which is why the classic design combines a hash map with a doubly linked list. The concurrency follow-up matters: maintaining exact global recency can become a lock bottleneck. Real high-throughput caches often approximate eviction rather than protecting one perfect global list.

#### How to judge the candidate

she names both structures and says why a list alone fails. Then ask about the concurrent version: a global lock removes the benefit, so real caches use segments or an approximation of the use order.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

### Q39. You must count unique visitors from a stream of one billion identifiers. Memory is limited. What do you do?

#### Reference answer

An exact set needs too much memory. Ask first if an approximate answer is acceptable. Usually it is. Options: Bloom filter: it answers "possibly present" or "certainly absent". It gives false positives but no false negatives. It uses a few bits for each item. It suits deduplication. HyperLogLog: it estimates the count of distinct items with a small, fixed memory, usually with an error near two percent. It suits a unique-visitor count. Exact and distributed: partition by a hash of the identifier, count on each partition, then add the results. This is correct but expensive.

#### Interviewer explanation - what this means

The first architectural question is whether the answer must be exact. HyperLogLog is appropriate for approximate cardinality with tiny fixed memory; Bloom filters answer membership, not cardinality, and can support deduplication. Reward candidates who clarify accuracy/error bounds before selecting a data structure.

#### How to judge the candidate

the first move is to ask about the accuracy requirement. That is the lead move. The tool choice comes second.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

### Q40. Explain consistent hashing and why a distributed cache needs it.

#### Reference answer

A simple method maps a key to a node with the remainder of the hash divided by the node count. If the node count changes, almost every key maps to a different node. The whole cache becomes invalid at one time. Consistent hashing places the nodes and the keys on a ring. A key belongs to the next node on the ring. When one node leaves, only its keys move. The other keys stay. Virtual nodes improve it. Each physical node takes many positions on the ring. This spreads the load and prevents one node from taking a large section.

#### Interviewer explanation - what this means

Modulo hashing causes massive remapping whenever the node count changes, creating a cold-cache event and downstream load spike. Consistent hashing limits remapping to a fraction of keys, and virtual nodes improve balance. The production insight is that cache topology changes can indirectly overload the database.

#### How to judge the candidate

she explains why remainder-based mapping fails during a scale event. That failure causes a database overload after a routine deployment.

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

### Q41. You must enforce 1,000 API requests per minute per customer across many service instances. Design the rate limiter.

#### Reference answer

The counter must be shared, because the instances must agree. A central store such as Redis holds it. Algorithm choice: Fixed window: a counter for each minute. It is simple. It allows a burst of twice the limit at the window

edge. Sliding window log: it stores each request time. It is exact and uses more memory. Sliding window counter: it mixes the current and the previous window. It is a good compromise. Token bucket: tokens refill at a fixed rate. It allows a controlled burst. This is the usual choice for an API. Other decisions: what the response is when the limit is reached (status 429 with a retry hint), whether the limit is for each customer or for each key, and what happens when the shared store is unavailable. A lead answers that the limiter must fail open or fail closed by a conscious decision, and states which one the business needs.

### **Interviewer explanation - what this means**

A distributed rate limiter needs shared or coordinated state so all application instances enforce one customer limit. Token bucket is common because it supports a sustained rate plus controlled bursts; fixed/sliding windows have different accuracy and memory trade-offs. A lead must also define 429 behavior, key scope, atomic updates and whether limiter-store failure should fail open or closed.

### **How to judge the candidate**

the failure behaviour of the limiter itself. Most candidates forget it. 9. Architecture scenario — client telemetry contract (main exercise, 20 minutes) This is a real system. Tell her so. Say: "We are building this now. I want your thinking. There is no single correct answer." This is honest, it removes the free-consulting feeling, and it makes her ask better questions. Grade the reasoning, not the design. You already made these decisions. Do not mark a different but defensible answer as wrong. Before the interview, divide your own decisions into two lists: forced by a constraint, and personal preference. Only the first list is gradeable. Scenario — give her this in two minutes, then stay quiet Client SDKs on web, iOS, and Android send telemetry to a Java backend. The backend is a gateway. It forwards error events to a log platform and derives metrics for a monitoring platform. The SDKs run in 300+ white-label targets. You cannot force a customer to upgrade. The web SDK sends batches. It uses a browser API that fires at page unload. Every event carries a severity: error, info, or debug. Task: design the contract between the clients and the gateway. Cover the request shape, the response, delivery guarantees, and change over time. Probes, in order. Do not give your answers. 1. "What do you need to know before you design this?" Strong: she asks whether the business accepts the loss of some logs. Telemetry usually tolerates loss. She must state that assumption, not assume it silently. She also asks about volume, event size, and who consumes the data. 2. "What does the server return, and at which moment?" The choice is to acknowledge before or after the downstream write. An early acknowledgement gives a fast client, and the gateway then owns the buffer and the risk. A late acknowledgement is safer and slower. Strong: she names the trade-off and picks one for a stated reason. 3. "One event in a batch of fifty is malformed. What happens?" Weak: reject the whole batch. Strong: accept the good events, count and report the bad one, and never lose 49 events because of one. 4. "The web SDK delivers at page unload. The response is not usable there." This is the best probe in the exercise. Client-side retry and unload-time delivery are incompatible. If she sees the conflict without help, she has built a client SDK. Follow with: "So where does the retry live?" Good answers: persist and send on the next page load, accept the loss for debug severity only, or use a different transport when the page is still alive. 5. "Do duplicates actually hurt you here?" Strong: she asks this before she designs deduplication. She may argue for deduplication at query time instead of at ingest time. If she deduplicates at ingest, she must name the key source (a client-generated event identifier), the window, the storage, and the cost of that state. 6. "What is the scope of the correlation identifier?" Options: one page load, one session, one SDK initialisation, or one flow across a parent SDK and its child SDKs. Ask also whether it joins to the server-side trace. This has no obvious answer. Her reasoning here is the highest-value part of the interview. 7. "Client clocks are wrong. Now what?" Strong: keep the client timestamp as reported, add a server receive time, and never order events by the client value alone. She may also mention a clock- offset field calculated at handshake. 8. "There are 300+ targets. Old SDK versions stay live for years. How do you change the contract?" This is the true architect question in this context. Expected: additive fields only, tolerate unknown fields on both sides, version the envelope, and measure traffic from old versions

before you remove anything. Weak: "we release version 2". 9. "One customer sends a hundred times the normal volume at 03:00." Strong: a rate limit for each key, severity-aware shedding (drop debug first, never drop errors), a payload size limit, and a remote switch delivered through configuration so you can stop a bad client with no release. 10. "What must never appear in these logs?" The payloads come from a fingerprinting SDK. Personal data is a real risk. A lead answers with an enforcement point — a schema check, a field allow-list, or a filter in the gateway — and not with a policy document. Scoring Strong: she asks about loss tolerance and volume before she designs. She names the acknowledgement trade-off. She finds the unload/retry conflict herself. She treats the 300+ targets as the main constraint on the contract. She says which parts she would defer. Adequate: the design is correct, but she handles only the successful path until you probe. She knows the patterns and applies them after a prompt. Weak: she starts with field names and JSON structure. She promises exactly- once delivery. She designs retries with no question about the transport. She assumes every client can upgrade. Note for you: probe 6 is an open question in your own design. A good answer here is useful to you. Keep your grading fair. Judge how she reaches the answer, not whether she reaches yours. 10. Lead behaviour Ask three of these. Follow each answer with "what happened after that".

**Lead-level discriminator:** Ask for a real production example, the constraint that drove the choice, and one downside or failure mode. A purely textbook answer is adequate technical knowledge, not yet lead-level evidence.

## 10. Lead behaviour

### Q42. Describe an architecture decision that you made and later reversed.

#### What a good answer should contain

a real reversal, the signal that told her, and how she managed the cost. A candidate with no reversal in 18 years is either not honest or was never the decider.

#### Interviewer explanation - what this means

This is not testing whether the candidate has ever been wrong; it tests whether they detect evidence, change course and manage sunk cost. A strong story has the original constraint, the signal that invalidated the decision, the reversal process and what was learned. Weak answers disguise a minor implementation tweak as an architectural reversal.

**Lead-level discriminator:** Look for a specific situation, the candidate's own decision/action, measurable consequences, and what changed afterward. Generic leadership language should score lower.

### Q43. A senior developer in your team disagrees strongly with your design. Describe a real case.

#### What a good answer should contain

she describes the technical argument of the other person correctly and fairly. She either changed her design or gave a clear reason and kept ownership. Bad sign: she won because of her title.

#### Interviewer explanation - what this means

You are testing technical humility and decision ownership. The candidate should represent the other engineer's argument fairly, use evidence or experiments where possible, and explain how the team reached closure. Authority can break a tie, but 'I was the architect' should not be the primary reasoning mechanism.

**Lead-level discriminator:** Look for a specific situation, the candidate's own decision/action, measurable consequences, and what changed afterward. Generic leadership language should score lower.

### Q44. How do you record architecture decisions?

#### What a good answer should contain

decision records, a short document with context, options, decision, and consequences. Also: where the team stores it, and whether anybody reads it. If she says "we discuss it in the meeting", the knowledge lives in one head.

#### Interviewer explanation - what this means

An Architecture Decision Record (ADR) preserves why a choice was made, alternatives considered and consequences accepted. The value is future context: new team members can understand why an apparently odd constraint exists. Strong answers include where ADRs live, how they are reviewed, and how superseded decisions are linked rather than silently overwritten.

**Lead-level discriminator:** Look for a specific situation, the candidate's own decision/action, measurable consequences, and what changed afterward. Generic leadership language should score lower.

#### Q45. How do you get time for technical debt when the business wants features?

##### What a good answer should contain

she converts debt into business language — incident rate, delivery speed, cost. She gives an example with a real number. Weak answer: "we ask for a debt sprint."

##### Interviewer explanation - what this means

Technical debt competes successfully when translated into business impact: incident risk, lead time, cloud cost, security exposure or developer capacity. Strong candidates quantify the effect and integrate debt work into normal prioritization rather than relying on occasional 'cleanup sprints'. Ask for a case where they chose not to repay debt because the economics did not justify it.

**Lead-level discriminator:** Look for a specific situation, the candidate's own decision/action, measurable consequences, and what changed afterward. Generic leadership language should score lower.

#### Q46. Describe your worst production incident. What did you change afterwards?

##### What a good answer should contain

a clear timeline, an honest role, a cause in the system and not in a person, and a specific change to the process. This is the single best question in this list.

##### Interviewer explanation - what this means

The best incident answer shows operational leadership under pressure and learning afterward. Look for timeline, detection, mitigation, root/system causes, communication and specific corrective actions with owners. Blaming one engineer is a warning sign; mature postmortems improve guardrails, tests, observability or process so the same class of failure is harder to repeat.

**Lead-level discriminator:** Look for a specific situation, the candidate's own decision/action, measurable consequences, and what changed afterward. Generic leadership language should score lower.

#### Q47. What do you look for in a code review?

##### What a good answer should contain

correctness, boundary handling, and clarity for the next reader. A lead also mentions what she does NOT comment on, such as style that a formatter can fix.

##### Interviewer explanation - what this means

A lead-level review prioritizes correctness, architecture/boundaries, failure handling, security, maintainability and testability. Formatting and mechanical style should be automated so human review attention stays on reasoning. Strong candidates also discuss review size, respectful feedback, risk-based depth and avoiding approval bottlenecks.

**Lead-level discriminator:** Look for a specific situation, the candidate's own decision/action, measurable consequences, and what changed afterward. Generic leadership language should score lower.

## 9. Architecture scenario - client telemetry contract

Use this as the highest-value exercise. The goal is not to make the candidate reproduce your design; it is to observe how they discover constraints, make trade-offs and protect a system that has long-lived client SDKs.

### Scenario to give the candidate

Client SDKs on web, iOS and Android send telemetry to a Java gateway. The gateway forwards error events to a log platform and derives metrics for a monitoring platform. The SDKs run in 300+ white-label targets, and customers cannot be forced to upgrade. The web SDK sends batches and can send at page unload. Each event has severity: error, info or debug. Design the client-to-gateway contract: request shape, response, delivery guarantees and evolution over time.

#### 1. What do you need to know before you design this?

Look for loss tolerance, volume, event size, downstream consumers, privacy/security constraints and transport behavior. The key signal is that the candidate asks questions before drawing a schema.

#### 2. What does the server return, and at which moment?

They should recognize acknowledgement timing as an ownership transfer. Early ACK improves client latency but means the gateway must durably buffer or consciously accept loss; late ACK couples the client to downstream latency/reliability.

#### 3. One event in a batch of fifty is malformed. What happens?

A robust telemetry design usually avoids throwing away 49 good events because one is bad. Look for partial acceptance, per-item error accounting, schema validation and metrics for rejected events.

#### 4. The web SDK sends at page unload and cannot rely on the response. Where does retry live?

This exposes whether the candidate understands browser/client constraints. Good answers may persist for the next load, accept controlled loss for lower severities, or use a different transport while the page is alive. Blind immediate retry is not credible at unload.

#### 5. Do duplicates actually hurt?

The candidate should first establish whether deduplication is worth its state and cost. If ingest dedup is proposed, ask for event ID generation, retention window and storage. Query-time dedup may be sufficient for telemetry.

#### 6. What is the scope of the correlation identifier?

There is no single correct answer. Judge whether they distinguish request/trace ID, page/session/SDK-instance correlation and cross-SDK business-flow correlation, and whether they can join client telemetry to server traces without creating uncontrolled cardinality.

#### 7. Client clocks are wrong. Now what?

Keep the reported client timestamp for context, add authoritative server receive time, and avoid ordering solely by client time. A sophisticated answer may discuss estimated clock offset.

#### 8. Old SDK versions remain for years. How does the contract evolve?

Look for additive evolution, tolerant readers, envelope/schema versioning, default semantics for missing fields and telemetry showing old-version usage before deprecation/removal.

## 9. One customer sends 100x normal volume at 03:00.

Look for per-customer limits, payload caps, severity-aware shedding, circuit/kill switches or remote configuration, and protection of shared downstream capacity. Error telemetry should generally be protected before debug traffic.

## 10. What must never appear in the logs?

A policy statement is insufficient. Strong answers add enforcement: explicit allow-lists/schema, redaction/filtering at the gateway, tests and monitoring for prohibited fields, with privacy/security ownership.

### How to score the scenario

Strong: asks about constraints before designing; names acknowledgement/loss trade-offs; independently spots the unload/retry conflict; treats long-lived client versions as a first-class compatibility constraint; explicitly designs overload and privacy controls; and states what can be deferred. Adequate: arrives at good patterns after prompting but initially designs only the happy path. Weak: jumps directly to JSON fields, promises exactly-once delivery, assumes clients can upgrade, or adds retries without considering the client transport and failure mode.

## Scorecard

| Area                           | Weight | Score 1-5 | Notes |
|--------------------------------|--------|-----------|-------|
| Java and JVM depth             | Medium |           |       |
| Concurrency                    | Medium |           |       |
| Spring internals               | Medium |           |       |
| Distributed systems            | High   |           |       |
| Data and persistence           | Medium |           |       |
| AWS design and cost            | High   |           |       |
| AI-assisted delivery judgement | Medium |           |       |
| Problem reasoning (DSA)        | Low    |           |       |
| Architecture scenario          | High   |           |       |
| Lead behaviour                 | High   |           |       |

Decision rule: a lead should score 4 or 5 in the high-weight areas. A gap in Java detail can be acceptable; a gap in trade-off reasoning, operational ownership or ability to explain decisions clearly is much more significant.